

一个事件只需一个词元：面向工业级大语言模型推荐的事件词元化

Fan Xia^{*,†}, Zhaoheng Zheng^{*}, Iman Setayesh^{*}, Ruogu Lin^{*}, Yiqin Pan^{*}, Samarth Mittal^{*}, Wentao Bao, Vinti Pandey, Sachin Patil, Jianpeng Cheng, Jun Xiao, Zhuang Wang, Xiangjun Fan, Sri Reddy, Minghai Chen

^{*} 核心贡献者, [†] 通信作者, Meta AI

基于 LLM 的推荐系统已沿模型容量和序列长度两个方向实现规模化, 但每个位置通常仅编码文本、语义 ID 或少量类别特征, 因而丢弃了每个事件中可用的丰富用户、物品、上下文和结果信号。在自回归建模下, 这会使每个位置的查询较弱; 由于每个位置又会成为下一位置的上下文, 性能退化将在序列中不断累积。我们提出一种以事件为中心的范式, 使用完整的时间快照表示每次交互, 并发现一个新的缩放维度, 称为**快照分辨率**: 即每个事件所编码的信息量。为了高效扩展快照分辨率, 我们提出 AMBER (通过瓶颈化事件表示进行自回归建模, Autoregressive Modeling via Bottlenecked Event Representation)。它将每个时间快照压缩为紧凑的**事件词元** (Event Token), 这是一种新的 LLM 输入模式。该表示以端到端方式学习, 而事件词元可预先计算并缓存以供在线服务使用, 从而将快照分辨率与实时服务计算解耦。在工业规模的排序和检索基准上, 相较其他推荐范式, AMBER 推进了计算—质量帕累托前沿。在容量充足时, 单个统一分词器甚至优于针对不同实体设计的专用分词器, 证明在结构不同的实体类型之间存在正迁移。AMBER 的事件词元还可以跨模型架构迁移: 将其作为服务时的历史特征集成到高度优化的非 LLM 排序模型后, 可带来具有统计显著性的提升。进一步扩大事件分词器容量还能获得额外改进。

日期: 2026 年 8 月 24 日

通信作者: Fan Xia, xiafanaifb@meta.com



1 引言

大型自回归模型正越来越多地用于处理现实世界事件的序列预测任务: 推荐模型对用户交互事件序列建模 [5, 24, 26], 世界模型根据观察—动作序列预测未来状态 [3, 11, 19], 金融模型根据金融事件序列预测市场走势 [23]。这些领域具有共同结构: 每个序列位置都编码一个**时间快照**——事件发生时系统观测的结构化记录——并由一组异构信号共同描述。我们将根据结构化事件序列预测未来状态的基础模型称为**大型事件模型 (LEMs)**。LLM 在固定词表上运行, 而 LEM 在现实世界事件的时间快照上运行。

LEM 与 LLM 的区别在于**快照分辨率**。任一时刻的真实系统状态都包含远多于系统可记录的信息; 快

照分辨率表示模型实际捕获的信息量, 即每个事件编码的不同信号数。例如, 推荐事件包含用户、物品、上下文和结果信号, 机器人控制事件包含视觉观测、关节状态和当前子目标。在自回归训练下, 每个位置更高的快照分辨率同时改善注意力两端: 查询表达更细粒度的意图, 键值历史为所有后续位置保留更多上下文细节。

工业推荐是大规模 LEM 的代表性实例, 受服务计算需求驱动, 目前面临序列长度与快照分辨率之间的严格权衡: 在大规模推荐系统中, 特征相关操作 (存储、反序列化和网络传输) 可能与 GPU 计算相当甚至更高。这一瓶颈迫使架构妥协。传统逐点模型 [6, 28] 为当前查询保留全面的异构特征, 却稀疏表示用户历

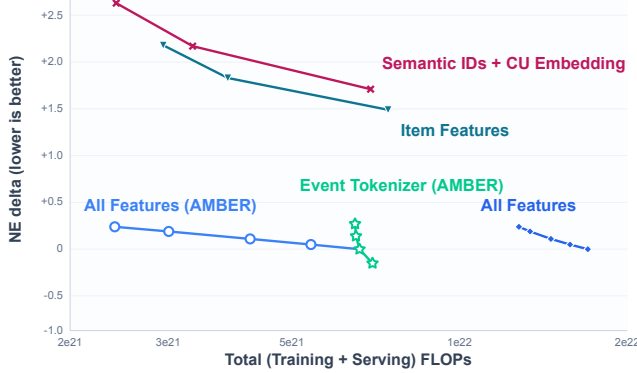


图 1: 在总训练与服务 FLOPs (包括 CPU 特征物化) 下扩展快照分辨率。从语义 ID 和内容理解嵌入, 依次扩展到物品特征, 再扩展到完整事件特征, 会逐步推进帕累托前沿。通过异步计算并缓存事件词元, AMBER 使完整特征扩展在实践中可行, 并使事件分词器扩展呈现出比用户 LLM 扩展更有利的计算—质量趋势 (第 5.4 节)。

史。近期自回归模型擅长序列建模, 却被迫将详细历史事件压缩为简单物品语义表示 [4, 5, 7, 24] 或人工筛选的特征子集 [12, 26]。两种情况下, 信息损失都会削弱单次预测; 由于每个位置又成为下一位置的上下文, 退化会在序列中逐步累积。

已有工作表明, 与 LLM 的嵌入空间对齐后, LLM 能够处理视觉词元等压缩潜在表示 [21, 22]。因此, 我们提出 AMBER (Autoregressive Modeling via Bottlenecked Event Representation), 一种以统一事件分词器为核心的架构。它并不将原始特征直接输入序列模型, 而是由分词器将每个时间快照中的海量异质信号封装为紧凑的事件词元, 再由下游序列模型以自回归方式处理。整个流程进行端到端训练, 学习一种高效压缩的事件词元表示, 并能随下游 LLM 容量有效扩展。在服务阶段, 由于词元化针对单个事件且与历史无关, 因此可以在事件发生时异步触发, 并将得到的事件词元缓存到用户的历史序列中 (图 2)。这样, 每个事件词元只需异步计算一次并缓存, 实时推理时便无需物化原始特征, 从而将快照分辨率与服务计算需求解耦。

我们验证了快照分辨率是工业推荐中可行且有效的缩放维度。在此基础上, 我们进一步证明事件词元范式具有以下特性, 使其非常适合大规模工业推荐:

第一, 事件分词器容量可以独立扩展, 从而改善下游性能, 同时对实时服务计算的影响极小。由于词元化针对每次曝光只触发一次并将结果缓存, 因此扩展事件分词器容量能够取得有利的计算—质量权衡。它可以显著提升下游质量, 却几乎不会增加主要在线服务路径的计算需求。

第二, 下游 LLM 在事件词元上的性能可随容量呈现可预测的缩放规律。将这些紧凑词元视为标准化的连续输入模态后, 随着参数容量增大, 下游序列模型的性能会单调提升。这表明事件词元并未遭受过度压缩, 而是保留了足够的细粒度语义信息, 能够成功释放下游模型的缩放定律。

除上述贡献外, 我们还分享了在大规模推荐系统中运行 AMBER 的实践经验, 重点讨论周期性表示漂移和 Event Token 存储。开放词表和不断演化的用户趋势要求定期重新训练分词器, 这会改变表示空间, 使新生成的 token 与缓存历史不兼容。我们采用对抗式周期对齐策略约束编码器更新之间的漂移, 从而缓解这种不稳定性。我们还应用 Matryoshka Dropout 和量化感知训练来减少 Event Token 的存储开销。

我们在覆盖排序与检索的大规模生产环境中评估 AMBER。在总训练与服务计算量约束下, 相较其他推荐范式, AMBER 推进了计算—质量帕累托前沿。通过将 Event Token 与 LLM 嵌入空间对齐, AMBER 使预训练权重优于随机初始化, 支持将 Event Token 作为新的 LLM 输入模态。大规模评估还表明, 现有非 LLM 排序器可直接将 Event Token 作为额外历史特征, 并获得统计显著的提升。

总而言之, AMBER 引入事件级词元化, 将异质特征纳入事件序列; 通过异步词元化实现计算高效的规模扩展; 并解决大规模应用中的表示漂移与存储挑战。

2 相关工作

推荐中的序列建模。工业推荐系统主要依赖两类特征: 逐曝光异构特征和历史行为特征。由于在线服务计算受到严格约束, 逐曝光查询的快照分辨率通常比历史事件高数十至数百倍。这种不对称迫使模型形

成严格的二分。**逐点模型** [6, 28] 使用高分辨率逐曝光特征对低分辨率历史进行注意力计算；虽然性能强劲，但每次曝光都重复编码历史，严重降低训练效率。相反，**自回归模型** [15, 26] 通过一次高效的因果前向过程按序处理历史，但各位置的查询快照分辨率天然较弱，最终限制整体性能。AMBER 弥合了这一差距：既具备自回归建模的单遍训练效率，又释放了逐点模型所具有的高逐事件分辨率。

LLM 推荐. 大多数基于 LLM 的推荐器将事件序列化为文本 [9]、Semantic IDs [4, 7, 12, 24]，或同时使用二者。然而，在严格服务约束下，有限的上下文窗口迫使系统在逐事件 token 预算与历史覆盖范围之间直接权衡。因此，两种范式都有所妥协：Semantic IDs 能高效编码物品，却丢失详细事件上下文；文本可以描述完整事件，但信息密度较低。与多模态 LLM [1, 21, 22] 类似，HLLM [5] 通过端到端训练将物品文本压缩为嵌入，但会舍弃相关的事件级信号。同期工作 LoopFM [14] 使用基础模型（FM）的中间表示构造序列；然而，这种解耦蒸馏阻断了下游梯度，当下游模型的容量与 FM 相当时，改进会减弱。AMBER 通过端到端训练一个以事件为中心的分词器，同时克服这两类问题：它将完整异构事件压缩为高密度 Event Tokens，并直接针对下游序列建模进行优化。

表示漂移与系统效率. 缓存学习到的表示会带来两项系统性挑战：周期训练期间的表示漂移，以及大规模场景下可观的存储需求。为解决时间漂移，我们借鉴领域自适应 [8, 10, 25]；该领域旨在对齐不同分布下的特征空间。尽管这类方法传统上用于跨领域任务，我们将其对抗式原理 [8] 改用于稳定连续时间检查点之间的表示。为解决存储瓶颈，我们利用表示压缩领域的最新进展，尤其是 Matryoshka Representation Learning [17, 18] 和量化感知训练 [2]，显著降低缓存 token 的内存占用。

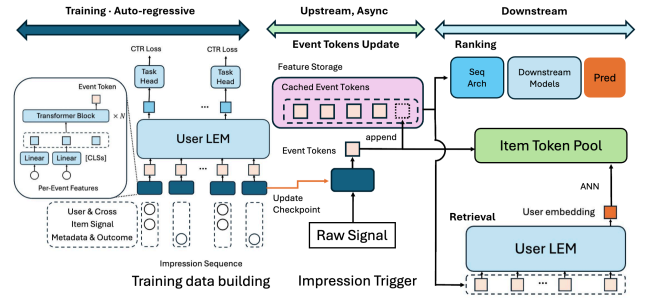


图 2: AMBER 概览。训练（左）：Event Tokenizer 在自回归目标下与 User LLM 端到端训练，学习将异构特征压缩为潜在嵌入（Event Token）。异步上游（中）：事件发生时，Event Tokenizer 将完整特征集压缩为紧凑的 Event Token，并追加到用户现有的 Event Token 序列；物品 token 也用于更新现有物品 token 池。下游（右）：排序和检索模型将缓存的 Event Token 序列作为序列特征。通过异步压缩完整事件特征，AMBER 无需在服务时物化完整特征集即可提高用户历史的快照分辨率，从而避免相应服务计算（第 3.1 节）。

3 方法

3.1 AMBER：模型架构

当前工业推荐系统在训练与服务之间存在显著的计算不对称。离线训练没有严格的延迟约束，并且只处理实际记录的行为以及合成负样本，因此其计算成本仅占在线服务的一小部分；后者需要实时为大型候选池打分。我们的架构利用这一结构性不对称，有意增加离线训练计算量，以降低在线服务的计算需求。

遵循这一理念，我们提出 **AMBER**：一种以事件为中心的范式，通过额外离线训练将每次交互的异构特征封装为紧凑 token，使下游模型在服务时使用缓存的 Event Token，而无需物化细粒度逐事件信号，从而以低得多的服务计算获得相当质量。它包含两个组件：

- **Event Tokenizer g_θ** ：将原始逐事件信号压缩为 d_z 维的 **Event Token z_i** 。
- **User LLM f_ψ** ：使用 Event Token 并自回归预测。

实验中, f_ψ 由预训练的 10 亿参数 Llama 模型初始化。RoPE 使用相对于固定参考日期的事件时间戳, 而非序列索引。训练期间, 两个模型 (g_θ 、 f_ψ) 联合优化。服务期间二者解耦: Event Tokenizer 异步运行, 下游模型根据缓存的 Event Token 序列预测 (图 2)。由于 User LLM 是仅解码器 Transformer, 本节其余部分重点介绍 Event Tokenizer。

Tokenizer 架构. Event Tokenizer 分三步将原始信号映射为 Event Token (图 7(a); 详见附录 B):

- (1) **特征编码.** 稀疏类别特征 (嵌入查找)、嵌入特征 (线性投影) 和稠密数值特征 (归一化、拼接并投影) 分别映射为独立的 d_{model} 维 token $\mathbf{h}_1, \dots, \mathbf{h}_m$ 。
- (2) **交互 & 汇总.** 向特征 token 追加 c 个可学习的 [CLS] token $\mathbf{h}_{\text{CLS}}^{(1)}, \dots, \mathbf{h}_{\text{CLS}}^{(c)}$, 使序列通过 L 层双向 Transformer 编码器, 让特征交互并汇总至 c 个 [CLS] 输出。
- (3) **LLM 空间投影.** MLP 将上下文化的 [CLS] 输出映射为最终的 d_z 维 Event Token。

具体而言, Event Token 为

$$\mathbf{z}_i = \text{MLP}(\text{BiTransformer}(\mathbf{h}_1, \dots, \mathbf{h}_m, \mathbf{h}_{\text{CLS}}^{(1:c)})[\text{CLS}]). \quad (1)$$

我们还在第 5 节与其他架构比较。

统一 Event Tokenizer. 不使用任务专属编码器, 单个共享 Event Tokenizer g_θ 通过动态特征掩码生成 token:

$$\mathbf{z}_i = g_\theta(\mathbf{f}_i, \mathbf{m}), \quad \mathbf{z}_i \in \mathbb{R}^{d_z} \quad (2)$$

其中, $\mathbf{f}_i$ 表示事件 e_i 的异构特征, $\mathbf{m}$ 是一个基于角色的二值掩码, 用于规定特征的可见性 (表 1)。除了简化服务之外, 该共享设计还通过实体之间的正迁移, 优于为每种实体专门设计的编码器 (§5.3.2)。

3.2 训练与序列设计

三阶段训练. 直接使用预训练 LLM 训练随机初始化的事件分词器, 会破坏 LLM 的世界知识。因此, 我

表 1: 按角色的特征掩码。所有角色共用一个 Event Tokenizer; 仅输入掩码不同。✓: 保留, ✗: 掩码 (元数据 = 位置、权重) (第 3.2 节)。

词元作用	用户	物品	上下文 & 交叉特征	元数据 & 结果
排序上下文 (A)	✓	✓	✓	✗
排序标签 (B)	✗	✗	✗	✓
检索历史	✓	✓	✓	✓
检索目标	✗	✓	✗	✗

们采用受视觉—语言模型训练启发的两阶段对齐 [21, 22], 随后进行循环训练:

阶段 1 (预对齐). 我们冻结用户 LLM, 仅训练事件分词器, 使事件特征与 LLM 的预训练嵌入空间对齐。

阶段 2 (联合训练). 我们解冻用户 LLM, 并以端到端方式训练两个组件, 使其表示共同适应。

阶段 3 (循环训练). 我们定期重新训练, 以跟踪分布偏移。第 4 节讨论更新过程中缓存事件词元产生的漂移。

序列设计. 上下文长度是序列推荐中的瓶颈: 在固定上下文窗口下, 每为一个事件增加一个词元, 就会挤出一个历史事件。实验 (第 5.3.2 节) 表明, 相比增加每个事件的词元数量, 优先扩大历史覆盖范围可以取得更好的性能。我们寻求各任务所需的最小词元预算, 因此检索任务每个事件使用一个词元, 排序任务每个事件使用两个词元 (图 3, 表 1)。

排序目标 (自回归 BCE). 对于每个事件, 在互补掩码下交错排列上下文 token $\mathbf{z}_i^{(A)}$ 和标签 token $\mathbf{z}_i^{(B)}$ (表 1), 形成序列

$$(\mathbf{z}_1^{(A)}, \mathbf{z}_1^{(B)}, \mathbf{z}_2^{(A)}, \mathbf{z}_2^{(B)}, \dots, \mathbf{z}_n^{(A)}, \mathbf{z}_n^{(B)}). \quad (3)$$

每个上下文位置 $\mathbf{z}_i^{(A)}$ 通过 MLP 头预测事件 i 的标签。由于标签 token $\mathbf{z}_i^{(B)}$ 在序列中位于 $\mathbf{z}_i^{(A)}$ 之后, 预测位置永远无法关注它, 从而避免泄漏。训练在单次前向传播中处理整个序列:

$$\mathcal{L}_{\text{rank}} = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)], \quad (4)$$

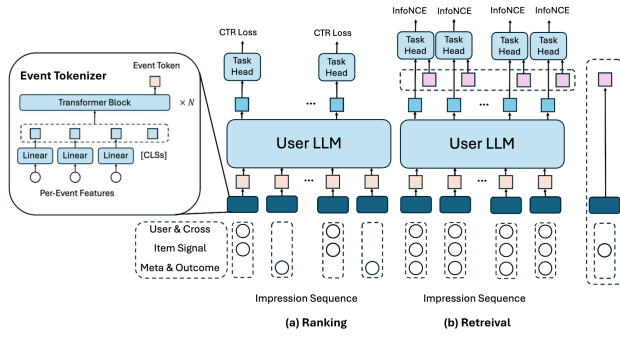


图 3: AMBER 用于两项任务。(a) 排序使用上下文 token 与标签 token 交错的序列及 BCE 目标。(b) 检索使用融合 Event Token 及 InfoNCE 目标；候选物品（粉色）单独 token 化。所有 Event Tokenizer 实例共享权重，仅针对不同 token 类型使用不同特征掩码（第 3.2 节）。

其中， $y_i \in \{0, 1\}$ 为行为标签。推理时，将候选物品作为不带标签的上下文词元附加到序列末尾。

检索目标 (InfoNCE)。我们将每个事件融合为单个不受掩码限制的词元，形成序列

$$(\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n), \quad (5)$$

从而最大化 LLM 所关注的历史范围。在每个位置, LLM 输出一个用户嵌入 $\mathbf{u}_i$ ；目标物品使用同一分词器，根据仅包含物品的特征预先编码，并独立于用户进行缓存。我们使用噪声对比估计损失进行训练：

$$\mathcal{L}_{\text{ret}} = -\frac{1}{n} \sum_{i=1}^n \log \frac{\exp(\mathbf{u}_i^\top \mathbf{v}_i^+ / \tau)}{\exp(\mathbf{u}_i^\top \mathbf{v}_i^+ / \tau) + \sum_{j \in \mathcal{N}_i} \exp(\mathbf{u}_i^\top \mathbf{v}_j^- / \tau)}, \quad (6)$$

其中， $\mathbf{v}_i^+$ 是用户近期未来发生正向交互的物品， τ 是温度参数， $\mathcal{N}_i$ 结合困难负样本（已展示但未交互的物品）与简单负样本（随机采样物品）。

4 大规模服务

4.1 异步上游架构

AMBER 采用两阶段系统，将特征计算与实时服务解耦：

离线 token 化与缓存。 Event Tokenizer 异步运行：当曝光事件发生时被触发，通过现有训练数据流水线查询该事件的完整异构特征集，并将其压缩为一个 Event Token。每个 token 都会追加到特征存储中的用户 Event Token 序列。

下游模式。 缓存的 Event Tokens 支持两种下游模式：

- 排序。使用端到端 LLM 进行实时排序会带来显著的基础设施瓶颈，尤其是长用户历史的 KV cache 维护问题。解决这些服务挑战超出了本文范围，但我们证明了该表示的通用可迁移性：缓存的 Event Tokens 可以作为即插即用的序列特征，提供给现有的非 LLM 排序模型（在 §5.6 中验证）。
- 检索。User LLM 直接使用缓存的 token 序列，生成实时用户嵌入，以进行近似最近邻 (ANN) 搜索。

4.2 缓解表示漂移

循环训练期间，将编码器从 $g_{\theta_{\text{old}}}$ 更新至 $g_{\theta_{\text{new}}}$ 会改变输出空间，造成随重复训练累积的表示漂移（图 4），使新生成的事件 token 与此前缓存 token 不兼容（附录 F 中的 t-SNE 可视化），并导致下游 LLM 预测不稳定（附录 G）。

为缓解表示漂移，我们将基于领域对抗神经网络 (DANN) 训练 [8] 的对抗正则化与指数移动平均 (EMA) 更新结合。

对抗正则化。循环训练期间，训练轻量判别器 D_ϕ ，区分 token 由旧编码器还是新编码器产生：

$$\mathcal{L}_{\text{adv}} = -\frac{1}{n} \sum_{i=1}^n [s_i \log D_\phi(\mathbf{z}_i) + (1 - s_i) \log(1 - D_\phi(\mathbf{z}_i))], \quad (7)$$

其中， $s_i \in \{0, 1\}$ 表示编码器版本。梯度反转层促使 $g_{\theta_{\text{new}}}$ 生成与 $g_{\theta_{\text{old}}}$ 无法区分的表示，并最小化联合目标 $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_{\text{adv}}$ 。我们使用较小的 λ ($\sim 5 \times 10^{-3}$)，因为更大的取值可能产生退化表示：虽然能欺骗判别器，却无法减少漂移。

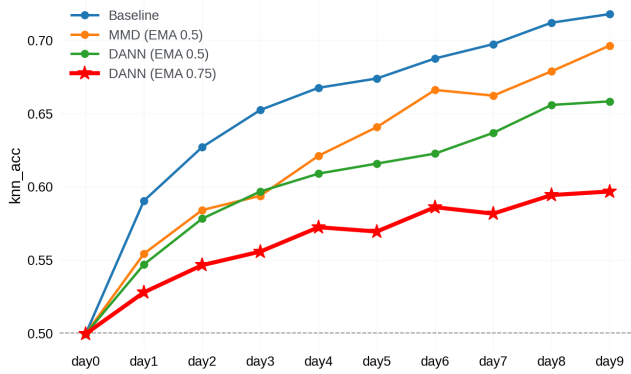


图 4: 随时间缓解表示漂移, 使用检查点可分性 (k -NN 准确率) 衡量; = 越高表示漂移越大, 0.5 表示 = 无法区分。无约束基线 (无正则化) 单调上升, 表明学习到的嵌入空间出现严重且持续累积的漂移。我们的稳定化配置 (EMA(0.5)+DANN、EMA(0.5)+MMD 和 EMA(0.75)+DANN) 抑制了这种分离, 使表示在连续重新训练周期中保持一致 (第 4.2 节)。

EMA 权重更新. 我们在每日训练周期之间维护一个 EMA 编码器, 作为 DANN 对齐每个新编码器的稳定参考, 而不是仅锚定到前一天的编码器。

我们在第 5.5 节将 DANN 与其他漂移缓解目标进行比较。

分片循环训练. 每天在完整用户群上重新训练十分浪费: 每个用户序列中只有一小部分是新增数据, 因此大多数数据会被重复处理。我们将用户随机划分为 N 个分片, 每天在一个分片上重新训练, 每 N 天遍历全部用户, 以最大化每次训练中的新鲜信号。模型收敛后, 我们发现这足以跟上最新趋势。

4.3 嵌入压缩

每个用户需要存储数百个预计算事件词元, 因此必须进行压缩以提高存储效率。我们采用两种互补技术。

套娃式 Dropout. 我们引入套娃式 Dropout (Matryoshka Dropout), 以支持灵活的词元维度, 而无需反复重新训练模型。标准套娃学习 [17] 要求每一步执行多次下游前向传播, 这对于 LLM 而言计算成本高得难以承受。我们改为通过结构化后缀 Dropout, 在单次前向传播中实现连续排序。以概率 ρ , 我们将 $\mathbf{z}$ 截断为前

$d_r \sim \text{Uniform}(d_{\min}, d_z)$ 个维度, 并应用 d_z/d_r 重缩放:

$$\tilde{\mathbf{z}}^{(j)} = \begin{cases} \frac{d_z}{d_r} \mathbf{z}^{(j)} & \text{若 } j \leq d_r, \\ 0 & \text{若 } j > d_r, \end{cases} \quad (8)$$

从而使模型学会在每个前缀中保留最重要的信息。

量化感知训练. 训练期间, 我们通过 STE [2] 使编码器接触模拟 INT8 噪声, 并在服务时使用 INT4, 在保留全精度嵌入影响的 $\sim 80\%$ 的同时, 实现 8 $\times$ 的存储缩减 (vs. FP32)。

5 实验

我们通过双轨验证评估 AMBER:

模型有效性与扩展性 (§5.2–§5.4). 我们将 AMBER 与高度优化的排序模型 (下称现有模型) 及其他范式比较 (§5.2), 对输入特征和 tokenizer 设计开展消融实验 (§5.3–§5.3.2), 并分析 tokenizer 容量、特征数量和下游模型规模的扩展行为 (§5.4)。

跨架构迁移 (§5.6). 我们将解耦 tokenizer 作为即插即用特征提供给现有非 LLM 排序器, 以证明 Event Token 与模型架构无关。

5.1 实验设置

5.1.1 数据集. 模型使用某大型工业推荐平台 PB 级曝光日志训练, 并按时间顺序划分: 训练和特征选择使用过去数据, 校准仅使用评估前的近期数据, 所有指标均在未见未来日期上计算。每个事件携带用户、物品、上下文和结果信号中数百个最重要特征; 消融实验 (§5.3–§5.3.2) 使用 10% 子样本, 主要结果 (§5.2) 使用完整数据集。

AMBER 面向特征物化、循环训练和非平稳性引起的现实系统约束。评估这些约束需要每个事件包含数百个异构特征及十亿规模时间数据。PB 级工业数据同时具备这两项性质, 常见公开推荐基准则没有。

5.1.2 指标. 排序的主要指标是归一化熵 (NE) [13], 这是工业推荐中广泛用作在线排序质量代理的标准离线

指标。NE 定义为平均对数损失除以经验正样本率 p 的熵：

$$NE = \frac{-\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]}{- [p \log p + (1 - p) \log(1 - p)]}, \quad (9)$$

其中, $y_i \in \{0, 1\}$ 是真实标签, p 是评估数据集中的经验交互率。所有 NE 改善均报告为相对差值 (NE Δ)。指标如下：

NE。NE 根据对数损失计算, 同时反映排序质量与概率校准：即使排序不变, 过度自信或信心不足也会增大 NE。因此, 我们在近期且无泄漏的数据上校准模型, 使总体预测交互率匹配观察交互率 (校准比接近 1), 再在未见未来数据上计算 NE。

集成 NE。直接跨范式比较不足以衡量互补预测收益。集成 NE 衡量 AMBER 相对于高度优化现有模型的互补收益。在未见未来数据上, 线性混合二者预测后计算 NE, 混合权重在前一个留出切片上调优。为考虑特征物化延迟, 屏蔽此前位于 **10 分钟**或 **1 分钟**延迟窗口内的事件。集成 NE 越低, 说明 AMBER 捕获的额外信号越多。

软召回率。对于检索, 我们报告按价值加权的最高价值物品保留比例：将模型候选与现有模型检索候选合并, 再由下游排序器重评分, 以衡量端到端效用而非孤立检索精度。

0.02% 范围内的 NE 差异取两次运行平均值；运行间波动至多为 0.02%。

5.1.3 基线.

现有模型。现有模型指在数万亿样本上训练的高度优化排序模型, 我们以其为参照报告集成 NE。现有模型 (公平) 在特征、FLOP 预算和训练目标上与 AMBER 匹配, 是 NE 的受控基线。

HSTU 风格输入。该基线遵循 HSTU 输入模式 (每个事件包含物品类别特征, 用户特征仅在序列开头出现一次), 保留 AMBER 的 LLM 以隔离输入模式影响；

表 2: 排序性能。NE Δ 以 Incumbent (公平设置) 为参照报告。Ensemble NE Δ 衡量将预测结果与 Incumbent 进行线性融合时所提供的互补预测信号, 并在模拟 10 分钟和 1 分钟特征延迟下评估 (第 5.2 节)。

方法	NE Δ ↓	集成 NE Δ ↓ (vs. 现有模型)	
	vs. 现有模型 (公平设置)	10 分钟延迟	1 分钟延迟
SID + CU emb	+1.20%	-0.03%	-0.05%
HSTU-style	+0.60%	-0.03%	-0.06%
Pointwise [†]	-0.10%	—	—
AMBER	-0.40%	-0.10%	-0.16%

[†] Pointwise 设置见附录 D。

已有工作发现 Transformer 与 HSTU 骨干在大规模下相当 [16] (附录 C) ¹。

语义 ID + CU 嵌入。消融实验 (§5.3.2) 表明, 在固定上下文长度下, 历史覆盖率的贡献大于增加每个事件 token 数。因此, 将多个语义 ID 与内容理解 (CU) 嵌入融合为单个 token。CU 嵌入是语义 ID 解压缩的无损上界。为控制 tokenizer 架构和容量, 该基线采用与 AMBER 相同的 Event Tokenizer 架构与规模。

逐点模型[†]。为区分自回归训练效率和历史分辨率, 该变体在 AMBER 骨干中复现逐点输入：当前查询使用完整事件, 历史仅保留物品特征。完整物品特征集作为强上界, 尽管在线物化如此多历史特征的计算成本很高 (附录 D)。

5.2 主要结果

5.2.1 排序。如 §5.1 所述, 我们同时评估受控模型质量和互补预测信号。表 2 表明, AMBER 在这两个方面均有改进。

与受控基线的性能比较。AMBER 取得最低的 NE, 分别比现有模型 (公平设置)、Pointwise、HSTU-style 输入以及 Semantic IDs + CU embeddings 高 0.40%、0.30%、1.00% 和 1.60%。相对 Semantic IDs + CU embeddings 的 1.60% 改进凸显了以物品为中心的表示所面临的信息瓶颈。AMBER 相对 Pointwise 的 0.30% 优势证实, 在整个历史中维持高快照分辨率——而非仅在当前查询处

¹下游序列模型架构不是 AMBER 的重点。Event Tokenizer 也可搭配 HSTU。为受控比较, 我们遵循现有 HSTU 输入模式, 同时使用与 AMBER 相同的 10 亿参数 LLM。

表 3: 自上次重新训练以来不同天数下, 检索 Soft Recall 的 Δ (相对现有模型, 见第 5.2 节)。

Soft Recall $\uparrow$	第 1 天	第 2 天	第 4 天	第 8 天
CU emb.	+0.23%	+0.09%	+0.20%	-0.01%
AMBER	+0.51%	+0.35%	+0.50%	+0.31%

维持——是有益的。同时, 在计算量匹配时, Pointwise 相对现有模型 (公平设置) 的 0.10% 改进分离出了自回归建模的效率收益。最后, 尽管 HSTU-style 输入借助静态用户侧特征和物品类别特征, 相对 Semantic IDs + CU embeddings 将 NE 降低了 0.60%, 但相近的集成 NE 表明, 这些信号与现有模型中的信号大多重复。

对现有模型的互补信号. 将 AMBER 与现有模型混合后, 在特征延迟 10 分钟时, 集成 NE 降低 0.10%; 在 1 分钟窗口下则降低 0.16%。相比之下, Semantic IDs + CU embeddings 仅带来 0.03% 和 0.05% 的降幅——不足 AMBER 改进的三分之一——且在没有 CU embeddings 时进一步恶化 (§5.3.1)。尽管存在这一差距, 以物品语义为基础的基线仍能带来非零的集成改进, 这可能部分源于 LLM 的容量。

5.2.2 检索. 我们通过 Soft Recall (§5.1) 评估检索。AMBER 在所有检查点年龄下均保持 0.31–0.51% 的改进, 比 CU embeddings 高 0.28–0.32 个百分点 (表 3)。后者到第 8 天已变为无增益, 而 AMBER 仍保持 0.31% 的改进, 这表明高分辨率事件信息能够改善检索, 并且对检查点陈旧具有稳健性。

5.3 消融研究

我们进一步对特征组和分词器架构进行消融, 以推导 AMBER 的设计选择; 随后开展同时考虑训练与服务计算的缩放分析 (§5.4)。

5.3.1 特征组消融. 我们进行两项互补分析: (1) 从完整 AMBER 中每次移除一个特征组; (2) 从最小化的 Semantic ID 表示出发, 逐步添加特征。所有实验均使用相同的编码器和预对齐设置。

表 4: 特征组消融 (预对齐)。SID 表示 Semantic IDs; 集成 NE 是在 10 分钟延迟下相对现有模型测得的。留一法结果按特征类型分组, 累积添加则从 SID + 结果信号开始 (第 5.3.1 节)。

配置	NE $\Delta \downarrow$	Ensemble NE $\Delta \downarrow$
	vs. 完整 AMBER	延迟 10 分钟
物品表示		
- 物品特征	+6.65%	-
- 物品类别信号	+1.00%	-
- 物品嵌入	+0.06%	-
- SID	无变化	-
- (SID + 物品嵌入)	+0.15%	-
- (SID + 内容哈希)	+0.07%	-
事件上下文		
- 用户特征	+0.30%	-
- 结果信号	+0.24%	-
- 元数据 (权重、位置)	+0.08%	-
- 用户—物品交叉特征	+0.03%	-
累积添加		
SID + 结果信号	+2.40%	-0.024%
+ CU 嵌入	+1.62%	-0.034%
+ 稀疏特征	+1.52%	-0.036%
+ 物品嵌入	+1.50%	-0.036%
+ 用户信号	+0.40%	-0.060%

物品信息很重要, 但语义 ID 并不充分. 移除全部物品特征会使 NE 恶化 6.65%, 证实物品表示仍是序列推荐中的主要信号。然而, 仅保留语义 ID 和结果信号时, 与完整 AMBER 相比仍有 2.40% 的差距。加入 CU 嵌入和非 CU 物品类别特征后, 该差距缩小至 1.52%, 说明仅靠语义 ID 无法捕获大量物品信息。

学习得到的物品表示所含信号大多重叠. 在 CU 嵌入和物品类别特征之上加入基于行为的物品嵌入, 只使 NE 改善 0.02% (1.52% $\rightarrow$ 1.50%)。类似地, 从完整 AMBER 中移除语义 ID 不会造成变化; 但同时移除语义 ID 与物品嵌入会造成 0.15% 的损失, 而仅移除物品嵌入的损失为 0.06%。这表明, 除学习得到的物品嵌入已经编码的信息外, 语义 ID 几乎不提供额外信息。

用户特征能够捕获行为序列之外的信号. 加入用户特征使 NE 改善 1.10%, 说明用户侧状态包含物品交互中不存在的信息。Ensemble NE 也印证了这一结果: 其最大提升来自用户信号和 CU 嵌入, 而二者在线物化的计算成本都很高; 相比之下, 将基于行为的物

品嵌入复用于事件历史时，几乎不提供额外信息。同时移除语义 ID 和内容哈希会产生 0.07% 的损失，这进一步表明不同标识符类型能够提供互补信息。

5.3.2 分词器 & 架构设计. 我们通过四项由数据驱动的消费实验验证 AMBER 的架构。

统一 *token* 化实现正迁移. 在四 *token* (用户、物品、其他、标签) 和双 *token* (上下文、标签) 模式下，共享 Tokenizer 均将 NE 改善了 0.02%，表明实体类型之间存在正迁移。

历史覆盖范围决定 *token* 预算. 在无限上下文下，每个事件使用两个 *token* 有所帮助 (0.10%)，但在有限窗口下反而有害 (0.16%)；因此 AMBER 优先保证历史覆盖范围。

Transformer 提供最佳实践权衡. 四层 *Transformer* 在相近 FLOPs 下与 DHEN ($\times 12$) 性能相当，同时具有更大的灵活性和 FlashAttention 效率；PMA 与 Concat+FFN (附录 B) 表现较差。

预训练知识可迁移到事件建模. 即使被冻结, LLM 相比仅使用 Event Tokenizer 仍提升 0.83%，表明直接将 Event Tokens 用作 LLM 输入具有潜力。解冻 LLM 还能额外提升 1.04%。尽管随着数据增多，预训练初始化的优势有所缩小，但其相对随机初始化的优势稳定在 0.20% (附录 H)。

5.4 效率 & 缩放分析

缩放方案. 我们在 {1, 2, 4, 8, 16} 范围内扩展 User LLM 深度，在 {1, 2, 4} 范围内扩展 Event Tokenizer 深度，在 {64, 128, 256, 512} 范围内扩展宽度，并在 {100, 200, 400} 范围内扩展快照分辨率。特征按置换重要性排序，该指标通过在样本间打乱每个特征后 NE 的增幅来衡量。我们在每种分辨率下均保留特征类型分布，并始终保留标签和元数据特征。

计算成本模型. 我们将总计算成本定义为

$$C_{\text{total}} = m_u C_{\text{user}} + m_e C_{\text{event}} + C_{\text{mat}}, \quad (10)$$

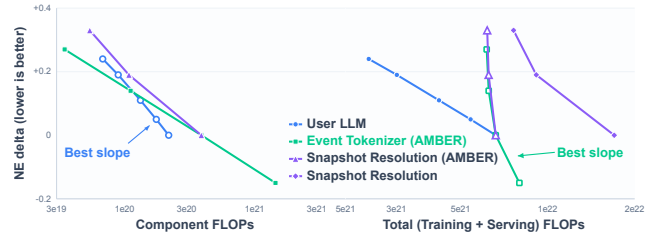


图 5: 仅计训练计算和计入总计算两种口径下的缩放。只计算训练 FLOPs 时，扩展 User LLM 更有利 (左)。计入服务 FLOPs 后 (右)，扩展 Event Tokenizer 和由 AMBER 实现的快照分辨率在计算上更高效，因为 *token* 化是异步执行的，其输出会被缓存 (第 5.4 节)。

排序工作负载的服务—训练比为 30–100 $\times$ ；我们采用下界。缓存 *token* 化对应 $m_e \ll m_u$ ， C_{mat} 刻画序列化。NE 为实测值，服务计算量为估算值 (附录 A)。

快照分辨率改善计算—质量前沿. 从语义 ID 和 CU 嵌入转向物品特征，再转向完整特征，NE 持续改善 (图 1)。这表明模型扩展无法恢复输入表示中缺失的信息。AMBER 将完整特征物化移出在线服务路径，进一步改善该前沿。

训练最优缩放并非系统最优缩放. 仅考虑训练 FLOPs 时，增加用户 LLM 容量呈现出最强的缩放趋势 (图 5, 左)。纳入服务计算后，事件侧缩放效率更高，因为用户 LLM 会针对每个请求运行，而事件词元只需计算一次并缓存 (图 5, 右)。因此，只依据训练计算量作出的缩放决策，在总计算量下可能并非最优。

Event Token 质量决定下游扩展性. 我们冻结由分别与 2 层或 16 层 User LLM 联合训练的 tokenizer 产生的 Event Token，并在未来数据上扩展新的下游 User LLM。与 16 层模型联合训练的 token 随下游计算增加而改善更快 (图 6)，表明更强联合训练产生的表示更便于大型下游模型利用。

5.5 漂移缓解评估

我们在相隔 20 天的检查点上比较 DANN 与 MMD、CORAL 和 L_2 正则化，这是一项更严格、对短期变化敏感度更低的漂移测试。余弦相似度比较不同编码器对相同事

表 5: AMBER 设计的消融实验。(a) 各训练阶段的贡献。(b) 跨实体类型共享单一分词器带来的迁移。(c) 每事件词元容量与历史覆盖范围之间的权衡。(d) 事件编码器比较 (第 5.3.2 节)。

(a) 训练		(b) 统一式 vs. 逐实体式		(c) 每事件的 token 数量		(d) 编码器	
设置	NE Δ ↓	模式	NE Δ ↓	上下文	NE Δ ↓ (每事件 2 vs. 1 个 token)	编码器	NE Δ ↓
仅 Tokenizer vs. 冻结 LLM	+0.83%	用户 / 物品 / 其他 / 标签		上下文		Bi-Transformer (×4, 基线)	—
解冻 vs. 冻结 LLM	−1.04%	(每事件 4 个 token)	−0.02%	无限上下文	−0.10%	Bi-Transformer (×2)	+0.09%
预训练 vs. 随机初始化	−0.20%	上下文 / 标签		有限上下文	+0.16%	Bi-Transformer (×1)	+0.19%
预对齐 vs. 直接联合训练	−0.10%	(每事件 2 个 token)	−0.02%			DHEN (×12) [27]	0.00%
						PMA (×4) [20]	+0.20%
						Concat+FFN	+0.22%

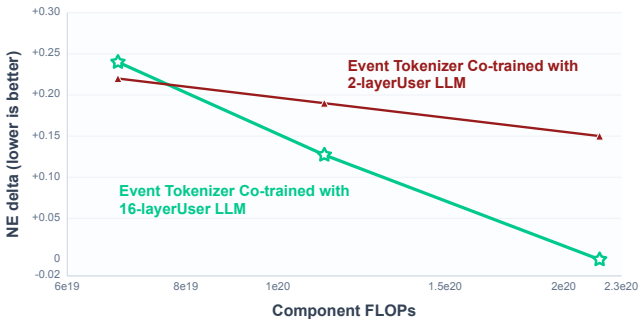


图 6: 使用冻结 Event Token 扩展下游 User LLM。我们使用分别与 2 层或 16 层 User LLM 联合训练的 Event Tokenizer 对未来数据进行 token 化, 冻结所得 token, 再扩展下游 User LLM。与 16 层模型联合训练的 token 呈现更强扩展趋势 (第 5.4 节)。

表 6: 相隔 20 天的检查点之间的漂移缓解效果。cos_mean 越高、knn_acc 越低表示对齐效果越好; NE Δ 以无正则化设置为参照 (第 5.5 节)。

方法	cos_mean ↑	knn_acc ↓	NE Δ (%) vs. 无正则化 ↓
DANN	0.956	0.856	−0.02
MMD	0.955	0.870	+0.03
CORAL	0.932	0.899	+0.36
L_2	0.900	0.939	+1.64

件生成的表示; k-NN 准确率则独立衡量非线性分类器区分这些表示的能力, 从而降低对所优化对齐目标的依赖。

DANN在不牺牲预测质量的情况下提供了最佳对齐, 其 NE 的轻微改善可能反映出模型对当前分布的过拟合有所减弱。

5.6 大规模评估

我们在一个全流量 Facebook 业务界面上集成 Event Tokenizer (384 万个稠密参数; ~200 GB 稀疏嵌入表), 支持每天对数十亿事件进行在线编码。我们通过 AMBER 的周期训练框架更新该模型, 并对每个记录事件仅执行一次 token 化, 以维护 Event Token 序列。现有模型将这些序列作为额外的历史特征使用。在内部数据集上, NE 降低 0.02% 即被视为具有统计显著性, 并足以带来可测量的收入影响。AMBER 的 Event Tokens 将 NE 降低了 0.06%。这一改进证明了跨架构迁移能力, 并表明提高历史快照分辨率能够提供现有模型中缺失的预测信号。

6 结论

我们证明, 提高历史事件的快照分辨率可以改善计算量—质量 Pareto 前沿, 但也会增加服务期间高昂的特征物化计算。为克服这一问题, AMBER 将异构事件特征异步压缩为紧凑的 Event Tokens。通过缓存这些表示, AMBER 无需在线特征物化即可使用高分辨率事件历史。

具体而言, AMBER 具有三项主要优势: (1) 与 LLM 进行端到端联合训练, 可实现高效的下游扩展; (2) 同时扩展 LLM 和 Event Tokenizer 后可以发现, 对系统级扩展决策而言, 优化训练与服务的总 FLOPs 比优化组件级训练 FLOPs 更为重要; (3) 领域对抗训练与 EMA 更新相结合, 可缓解周期服务期间的表示漂移。此外, 大规模评估取得了 0.06% 的 NE 改进。

未来工作. 我们证明, 下游性能会随 Event Tokenizer 容量扩展; 未来将探索混合专家 (MoE), 以实现更高

效的扩展。在服务侧，异步 token 化会引入一定的特征延迟（附录 E）。一个有前景的方向是维护保留时间极短（例如 10 分钟）的高分辨率序列，从而按需对尚未缓存的事件进行 token 化。最后，由于 Event Tokens 与 LLM 嵌入空间对齐，它们可以与文本联合建模，实际上相当于将用户视为一种新的 LLM 模态。我们计划沿此方向探索个性化 LLM，将语言理解与推荐信号结合起来。

References

- [1] Jean-Baptiste Alayrac et al. 2022. Flamingo: a Visual Language Model for Few-Shot Learning. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*. Curran Associates, Inc., 23716–23736. arXiv:2204.14198
- [2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *CoRR* abs/1308.3432 (2013). arXiv preprint. arXiv:1308.3432
- [3] Jake Bruce et al. 2024. Genie: Generative Interactive Environments. *CoRR* abs/2402.15391 (2024). arXiv:2402.15391
- [4] Junyi Chen et al. 2025. LCRec: Towards Generative Recommendation with Language-Based Collaborative Representations. *CoRR* abs/2502.14927 (2025). arXiv:2502.14927
- [5] Junyi Chen, Lu Chi, Bingyue Peng, and Zehuan Yuan. 2024. HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling. *CoRR* abs/2409.12740 (2024). arXiv:2409.12740
- [6] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior Sequence Transformer for E-commerce Recommendation in Alibaba. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data (DLP-KDD)*. ACM. arXiv:1905.06874
- [7] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. *CoRR* abs/2502.18965 (2025). arXiv:2502.18965
- [8] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2016. Domain-Adversarial Training of Neural Networks. *Journal of Machine Learning Research* 17, 59 (2016), 1–35.
- [9] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5). In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*. ACM, New York, NY, USA, 299–315.
- [10] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. 2012. A Kernel Two-Sample Test. *Journal of Machine Learning Research* 13 (2012), 723–773.
- [11] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. 2023. Mastering Diverse Domains through World Models. *CoRR* abs/2301.04104 (2023). arXiv:2301.04104
- [12] Ruining He, Lukasz Heldt, Lichan Hong, Raghunandan H. Keshavan, Shifan Mao, Nikhil Mehta, Zhengyang Su, Alicia Tsai, Yueqi Wang, Shao-Chuan Wang, Xinyang Yi, et al. 2025. PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations. *CoRR* abs/2510.07784 (2025). arXiv:2510.07784
- [13] Xinran He, Junfeng Pan, Ou Jin, Tianbing Xu, Bo Liu, Tao Xu, Yanxin Shi, Antoine Atallah, Ralf Herbrich, Stuart Bowers, and Joaquin Quiñero Candela. 2014. Practical Lessons from Predicting Clicks on Ads at Facebook. In *Proceedings of the Eighth International Workshop on Data Mining for Online Advertising (ADKDD)*. ACM, 5:1–5:9. doi:10.1145/2648584.2648589
- [14] Shali Jiang, Hua Zheng, Boyang Liu, Laming Chen, Huayu Li, et al. 2026. LoopFM: Learning from Historical Representations of Foundation Model for Recommendation. *CoRR* abs/2605.29280 (2026). arXiv:2605.29280
- [15] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *IEEE International Conference on Data Mining (ICDM)*. 197–206.
- [16] Kirill Khrylchenko, Artem Matveev, Sergei Makeev, and Vladimir Baikalov. 2026. Scaling Recommender Transformers to One Billion Parameters. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. arXiv:2507.15994
- [17] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Sapra, Prateek Jain, Ali Farhadi, and Sham Kakade. 2022. Matryoshka Representation Learning. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*. Curran Associates, Inc., 30233–30249. arXiv:2205.13147
- [18] Riwei Lai et al. 2024. Matryoshka Representation Learning for Recommendation. *CoRR* abs/2406.07432 (2024). arXiv:2406.07432
- [19] Yann LeCun. 2022. A Path Towards Autonomous Machine Intelligence. *OpenReview* (2022). <https://openreview.net/pdf?id=BZ5a1r-kVsf>
- [20] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. 2019. Set Transformer: A Framework for Attention-based Permutation-Invariant Input. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*. PMLR, 3744–3753. arXiv:1810.00825
- [21] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 19730–19742. arXiv:2301.12597
- [22] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. Curran Associates, Inc., 34892–34916. arXiv:2304.08485
- [23] Dat Mai. 2024. StockGPT: A GenAI Model for Stock Prediction and Trading. *CoRR* abs/2404.05101 (2024). arXiv:2404.05101
- [24] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Mahesh Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. Curran Associates, Inc., 10935–10959.
- [25] Baochen Sun, Jiashi Feng, and Kate Saenko. 2016. Return of Frustratingly Easy Domain Adaptation. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI)*. 2058–2065.

- [26] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. *CoRR* abs/2402.17152 (2024). arXiv:2402.17152
- [27] Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, Yang Liu, Huayu Li, Yasmine Badr, Jongsoo Park, Jiyan Yang, Dheevatsa Mudigere, and Elie Wen. 2022. DHEN: A Deep and Hierarchical Ensemble Network for Large-Scale Click-Through Rate Prediction. *CoRR* abs/2203.11014 (2022). arXiv:2203.11014
- [28] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*. ACM, 1059–1068. arXiv:1706.06978

A 总计算成本与敏感性分析

我们在一个共同评估周期内对排序系统进行建模。训练使用记录的曝光，大量采样负样本，并且仅保留一小部分考虑事件。相比之下，排序推理会在每个请求上运行，不进行下采样，并为传递到排序阶段的每个候选选项打分。由此得到如下工作负载分解：

$$m_u = 1 + r_{\text{traffic}} \frac{K_{\text{serve}}}{K_{\text{train}}} r_{\text{eff}}, \quad (11)$$

其中， r_{traffic} 是服务曝光量与保留训练曝光量之比， $K_{\text{serve}}/K_{\text{train}}$ 刻画全候选评分相对采样训练样本的放大， r_{eff} 考虑受延迟约束的推理相较批量训练利用率更低。工作负载分析表明， m_u 的数量级约为 30–100×；主要图表均采用下界。该范围专用于排序，并非全行业常数；检索工作负载的放大通常更有限。

图中根据整个训练过程的活跃参数 FLOP 计算 C_{user} 和 C_{event} ；稀疏嵌入查找不计入 FLOP。AMBER 异步调用 Event Tokenizer 并缓存输出，因此 $m_e \ll m_u$ 。未缓存的完整特征序列则令 $m_e = m_u$ ，反映排序路径上重复的事件侧计算。

特征物化计算取决于服务栈。我们利用特征服务与 GPU 服务之间的经验功率比，将其映射到统一 FLOP 坐标轴。由于该比率依赖基础设施，只报告粗略范围

而非绝对测量值。其一阶扩展关系为

$$C_{\text{mat}} \propto N_{\text{served}} H \sum_g n_g c_g, \quad (12)$$

其中， N_{served} 是已服务排序请求的数量， H 是物化历史长度， n_g 是每个事件中组 g 的特征数量，而 c_g 是在共同计算坐标轴上通过经验校准得到的逐值物化计算量。这些系数在不同服务栈之间可能有所不同。

对于共同计算坐标轴，我们将完整异构事件物化归一化为 1，并通过经验校准每种模式。Semantic IDs 与 CU embeddings 的计算量仅占这一参考计算量的低个位数百分比；物品特征则处于高个位数百分比范围，因为它们包含更多数值和更大的浮点负载。AMBER 使用固定大小的缓存 token 替代二者。这些范围仅影响横向计算坐标；所有 NE 值均为直接测量所得。

B 基线编码器架构

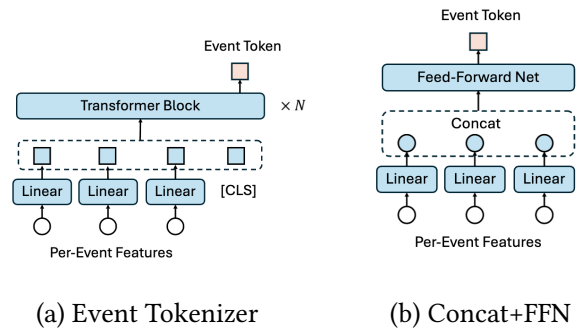


图 7: 事件编码器架构。(a) Event Tokenizer 使用 N 层双向 Transformer 处理投影后的逐事件特征和可学习的 [CLS] token，再将 [CLS] 输出映射为 Event Token。(b) Concat+FFN 直接拼接投影特征，并应用不含自注意力的前馈网络。

图 7 将 Event Tokenizer 与编码器架构研究 (§5.3.2) 中的 Concat+FFN 基线比较。Concat+FFN 不使用自注意力，并将特征间交互限制在前馈网络中。

表 7 给出比较结果。DHEN ($\times 12$) 在 FLOP 相当时与四层双向 Transformer 相当，PMA 和 Concat+FFN 的 NE 分别落后 0.20% 和 0.22%。选择 Transformer 是因为它更灵活，并可通过 FlashAttention 获得略高的实

表 7: 编码器架构比较。NE Δ 相对于默认四层双向 Transformer；正值表示退化。

编码器	NE Δ ↓
双向 Transformer ($\times 4$)	—
双向 Transformer ($\times 2$)	+0.09%
双向 Transformer ($\times 1$)	+0.19%
DHEN ($\times 12$) [27]	0.00%
PMA ($\times 4$) [20]	+0.20%
Concat+FFN	+0.22%

际效率。将其缩小至 $\times 2$ 和 $\times 1$ 时，NE 分别增加 0.09% 和 0.19%。

C HSTU 风格输入基线

AMBER 的 Event Tokenizer 与下游模型无关：它既可以向 HSTU-style 序列主干提供输入，也可以向 User LLM 提供输入；先前研究发现，这些主干在大规模场景下表现相当 [16]。因此，与 HSTU 有信息量的比较应针对输入模式，而非架构。该消融保持 AMBER 的 User LLM 主干不变，仅将输入模式改为与 HSTU 相同：每个事件只保留物品相关的类别特征，而用户侧特征只在序列开头提供一次，不再逐事件编码。为避免使其处于不利地位，我们为这些用户侧输入提供了宽松设置，包括所有用户浮点特征、嵌入特征和 ID 列表特征（参考实现仅在序列开头放置少量用户类别特征）。相对 AMBER，该模式的 NE 高 1.0% (+0.6% vs. 现有模型（公平设置））。

D 逐点基线

该基线将“在整个历史序列中保持高快照分辨率”的收益，与自回归建模带来的训练效率收益区分开来。若直接将 AMBER 与传统逐点架构比较，这两种效应会彼此混杂，因为自回归训练只需一次前向传播即可处理完整序列，而逐点训练需要针对每次曝光反复编码历史。因此，我们在相同的用户 LLM 主干内复现逐点输入模式，同时保留单次前向传播的自回归训练。

每个事件由三个词元表示：包含完整事件特征的高分辨率查询词元 A、包含完整物品特征集合的仅物

品历史词元 B，以及结果词元 C。尽管 B 的分辨率低于完整事件，但它仍是一个很强的上界；若在整个在线历史中物化其所含数量的特征，计算成本依然很高。注意力掩码强制实施逐点信息流。对于事件 i ， A_i 只能关注之前的 B 和 C 词元，因此其预测是在仅物品历史上使用高分辨率当前查询得到的。B 和 C 均不能关注任何 A 词元。这一限制防止高分辨率信息进入 B 或 C，并间接传播至未来预测；因此，持久化历史始终严格只包含物品信息。

该构造使主干网络、优化目标、计算预算和自回归训练效率均与 AMBER 保持一致，唯一变化是历史中是否保留高分辨率事件信息。逐点方法相较 Incumbent（公平设置）将 NE 改善 0.1%，从而分离出自回归训练的收益，但仍落后 AMBER 0.3%。因此，剩余差异支持了在整个历史中保持高快照分辨率的优势，而非仅在当前查询中使用高分辨率。

E 对特征物化时延的鲁棒性

在真实服务环境中，事件发生与其特征物化之间通常存在 1–5 分钟的延迟。我们在训练期间随机掩蔽某个时间窗口内的近期事件，以模拟这种延迟。与预期相反，即使评估时计入相同偏移，延迟模拟仍使 NE 恶化 0.2%。这表明模型本身对特征物化时延具有鲁棒性，而人工掩蔽反而移除了有用的训练信号。

F 表示漂移可视化

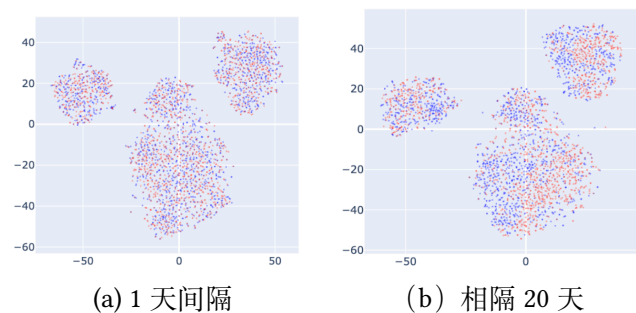


图 8: 两个编码器检查点 (红色 vs. 蓝色) 对相同事件编码所得事件词元的 t-SNE 可视化。(a) 相隔 1 天时, 两种分布充分混合。(b) 相隔 20 天时, 簇内出现颜色分离, 表明漂移不断累积。

H 预训练初始化动态

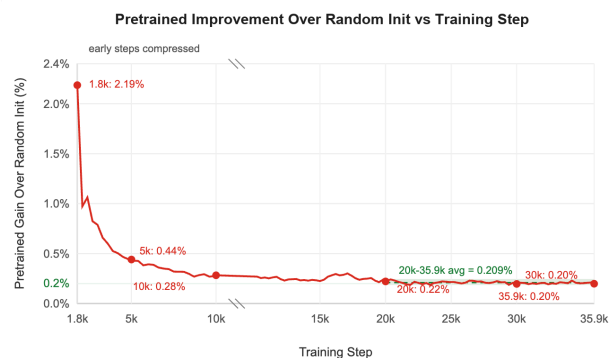


图 10: 预训练 LLM 初始化相较随机初始化的 NE 优势随训练步数的变化。该优势从训练初期的 $\sim 2.2\%$ 逐渐衰减, 并在 20K 步后稳定于 $\sim 0.2\%$ 。

G 循环训练稳定性

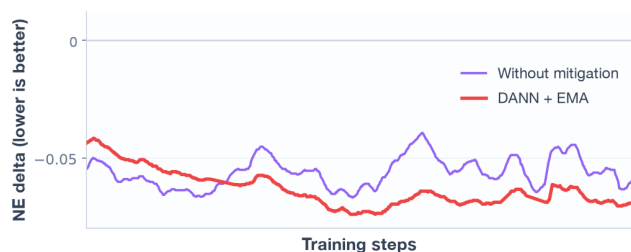


图 9: 周期训练期间的 NE 轨迹。不采用漂移缓解时 (紫色), NE 在不同训练步之间呈现较高方差; 使用 DANN + EMA 时 (红色), 更平滑的轨迹表明下游预测更稳定。两次运行使用完全相同的超参数。